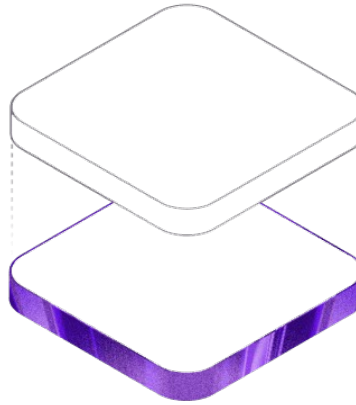


GAMEPLAN TASK BOARD

Software Design Document

Full-stack Kanban task management platform



Prepared by

Manan Hiteshkumar Patel

Repository: github.com/Mananp2207/gameplan-task-board

Live application: gameplan-task-board.vercel.app/login

Document version: 1.0 | August 2026

Implementation-grounded documentation

Verified against the uploaded source archive and project assessment. Planned capabilities are labeled separately from implemented behavior.

Document Control

Item	Value
Document owner	Manan Hiteshkumar Patel
Project	GamePlan Task Board
Document type	Software Design Document
Primary audience	Recruiters, interviewers, instructors, and future contributors
Source basis	Uploaded source archive, project assessment, and repository structure
Verification boundary	Static source review; database policy SQL and automated test suite are not included in the repository
Status	Portfolio-ready technical documentation with explicit limitations

Accuracy Statement

This document distinguishes between three categories: verified implementation, repository-indicated design, and future work. A capability is described as implemented only when it is visible in the uploaded source or explicitly confirmed by the assessment. Database Row Level Security is referenced because application comments and the assessment rely on it, but the SQL policy definitions are not present in the uploaded repository and therefore are not reproduced or claimed in detail.

Table of Contents

- • 1. Executive Summary
- • 2. Product Context and Problem Definition
- • 3. Scope, Stakeholders, and Requirements
- • 4. Verified Feature Inventory
- • 5. Technology Stack and Engineering Rationale
- • 6. High-Level System Architecture
- • 7. Frontend Architecture
- • 8. Routing and Navigation
- • 9. State Management
- • 10. Authentication and Profile Management
- • 11. Task Domain Model and Business Rules
- • 12. Task Data Access and Persistence
- • 13. Team Collaboration Architecture
- • 14. Database Model
- • 15. Authorization and Security
- • 16. User Experience and Accessibility
- • 17. Search, Calendar, Theme, and Keyboard Interaction
- • 18. Error Handling and Reliability

- • 19. Deployment and Configuration
- • 20. Testing and Quality Strategy
- • 21. Performance and Scalability
- • 22. Design Decisions and Trade-offs
- • 23. Challenges and Lessons Learned
- • 24. Known Limitations and Technical Debt
- • 25. Future Roadmap
- • 26. Developer Guide
- • 27. User Guide
- • 28. Interview Guide
- • Appendices A-F

1. Executive Summary

GamePlan is a full-stack task management application that organizes work through a four-stage Kanban workflow: To Do, In Progress, In Review, and Done. The uploaded implementation uses React and TypeScript on the client, Vite for development and production builds, Tailwind CSS for styling, React Router for client-side navigation, and Supabase for authentication, PostgreSQL data access, and avatar storage. The application is deployed through Vercel according to the project assessment.

The product is designed for individuals and small teams that need a lightweight way to create, prioritize, assign, review, and complete tasks. Its differentiating behaviors include anonymous guest access, Member and Supervisor roles, automatic ordering by urgency and due date, supervisor return messages, team membership management, a calendar view, search, light and dark themes, and keyboard shortcuts.

The architecture is intentionally pragmatic: the React client communicates directly with Supabase through the official JavaScript SDK instead of a custom server. This reduces infrastructure overhead while retaining a relational data model and a database-level authorization boundary. The source expects Row Level Security to decide which task rows a user can read or modify; however, policy SQL is not included in the repository, so this document describes the intended boundary without inventing policy definitions.

The repository also contains clear limits. The Analytics route renders a coming-soon page, no automated test suite is included, and drag-and-drop packages are installed but not imported by the source. These items are documented as planned or incomplete rather than represented as finished functionality.

2. Product Context and Problem Definition

2.1 Problem

Small teams often manage work through disconnected messages, spreadsheets, or simple checklists. These tools make it difficult to see task state, identify urgent deadlines, assign responsibility, and separate work awaiting review from work that is actually complete.

2.2 Proposed Solution

GamePlan provides one visual workspace in which users can create tasks, assign deadlines, mark urgent work, move tasks through a defined workflow, and organize collaboration through teams. The calendar complements the status-oriented board by presenting tasks around due dates.

2.3 Problems Addressed

- • Fragmented task visibility across different tools.
- • Manual and inconsistent priority decisions.
- • Unclear distinction between active, review-ready, and completed work.
- • Insufficient ownership and assignment context.
- • High onboarding friction for evaluators; anonymous guest access reduces this barrier.
- • Lack of a scalable foundation for Member and Supervisor responsibilities.

2.4 Product Value

The product demonstrates both practical utility and software-engineering depth. It combines a polished user interface with authentication, relational data access, role-aware workflows, team management, persistent user profiles, and deployment. For a portfolio, it provides a concrete system through which architecture, security, database design, and trade-offs can be discussed.

3. Scope, Stakeholders, and Requirements

3.1 Stakeholders

Stakeholder	Needs
Individual user	Create and track personal work with minimal setup.
Team member	View assigned tasks, update progress, and submit work for review.
Supervisor	Create/assign work, review tasks, return work with a message, and manage teams.
Evaluator or recruiter	Access the product quickly through guest authentication and understand the architecture.
Future contributor	Clear modules, types, services, setup instructions, and known limitations.

3.2 Functional Requirements

- Register and sign in with email and password.
- Create an anonymous guest session.
- Protect application routes from unauthenticated access.
- Load a persistent profile for authenticated non-guest users.
- Create, read, update, move, return, and delete tasks.
- Represent the workflow with four statuses.
- Order tasks using urgency, due date, and creation time.
- Search task title, description, and manager message.
- Create teams and manage team membership and roles.
- Display tasks in board, filtered status, home summary, and calendar views.
- Persist light/dark theme selection.
- Support keyboard shortcuts for theme, task creation, and modal dismissal.
- Support avatar color and image operations for permanent users.

3.3 Non-functional Requirements

- Maintainability through typed models and feature-oriented components.
- Security through managed authentication and database authorization policies.
- Responsiveness across common desktop and smaller viewports.
- Usability through clear status labels, loading states, validation, and modal workflows.
- Deployability through a static Vite build and environment-based configuration.
- Extensibility for analytics, notifications, comments, attachments, and realtime updates.

4. Verified Feature Inventory

Capability	Status	Evidence / qualification
Email sign-up and sign-in	Implemented	AuthContext calls Supabase signUp/signIn; login and sign-up routes exist.
Anonymous guest access	Implemented	Supabase signInAnonymously is used.
Protected routes	Implemented	ProtectedRoute wraps all non-authentication routes.
Four-stage Kanban workflow	Implemented	Status type: todo, inprogress, inreview, done.
Task CRUD and movement	Implemented	Task service exposes create, update, move, return, and delete.
Automatic ordering	Implemented	Queries order urgent first, then due date, then creation time.
Supervisor review/return	Implemented in source design	canReview derives from supervisor profile; return message moves task to To Do.
Search	Implemented	Client filters title, description, and manager message.
Calendar view	Implemented	CalendarPage is defined in App.tsx.
Teams and memberships	Implemented	Teams service supports create, add/remove member, role update, and delete.
Theme persistence	Implemented	Theme stored in localStorage and initialized from system preference.
Keyboard shortcuts	Implemented	D toggles theme, N opens create modal, Escape closes modals.
Avatar color and upload services	Implemented at service/context level	AuthContext validates and uploads JPG/PNG/WebP up to 2 MB.
Analytics dashboard	Planned / placeholder	Route renders ComingSoonPage.
Drag-and-drop board	Not verified	dnd-kit dependencies exist but no dnd-kit imports appear in src.
Automated tests	Not present	No test scripts or test files in uploaded repository.
RLS policy definitions	External to repository	Code relies on RLS; SQL migrations/policies are not included.

5. Technology Stack and Engineering Rationale

Technology	Role	Rationale
React 19	UI framework	Component composition and declarative rendering.
TypeScript 5	Type safety	Domain models, safer refactoring, and clear service contracts.
Vite 8	Build tooling	Fast local development and static production bundles.
Tailwind CSS 4	Styling	Responsive utility classes and consistent visual language.
React Router 7	Navigation	Public authentication routes plus protected nested application routes.
Supabase JS 2	Backend client	Authentication, PostgreSQL queries, and Storage access.
PostgreSQL	Persistence	Relational modeling for profiles, tasks, teams, and memberships.
React Hook Form + Zod	Form dependencies	Installed for structured form handling and validation; usage should be verified per form.
Lucide React	Icons	Consistent iconography.
Vercel	Hosting	Static frontend deployment and route fallback configuration.
Oxlint	Linting	Fast source analysis through npm run lint.

5.1 Backend-as-a-Service Decision

A managed backend avoids implementing password storage, token issuance, session refresh, database APIs, storage, and deployment as separate services. This is appropriate for the current product scale and portfolio objective. The trade-off is tighter coupling to Supabase APIs and security configuration.

6. High-Level System Architecture

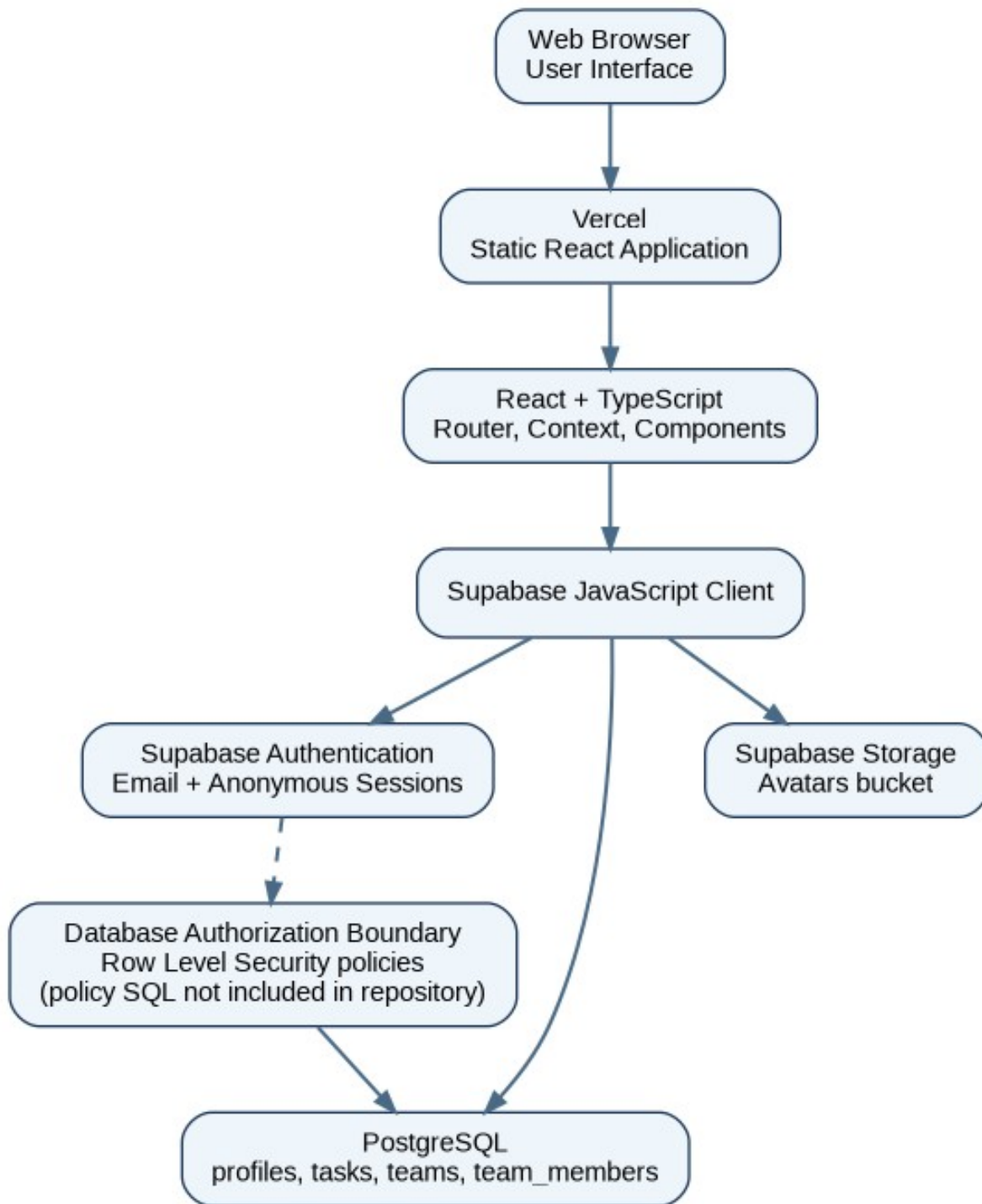


Figure 1. Verified high-level architecture and authorization boundary.

The browser loads the Vite-generated React application from Vercel. The React client manages presentation, routing, shared authentication state, and user interaction. Data and identity requests are issued through the Supabase JavaScript client. Supabase Authentication manages email and

anonymous sessions; PostgreSQL stores product data; Storage is used by avatar operations. Database policies are expected to enforce row-level authorization.

6.1 Architectural Style

GamePlan is a client-centric single-page application with managed backend services. It is not a traditional three-tier system with a custom API server. Service modules create a logical data-access layer inside the client, while the actual trust boundary remains in Supabase Authentication and PostgreSQL authorization policies.

6.2 Trust Boundaries

- Browser input is untrusted and must be validated.
- The Supabase anonymous key is public by design and does not replace RLS.
- Route protection improves navigation but cannot secure database rows.
- Database policies are the decisive authorization boundary.
- Avatar file type and size checks exist in the client; Storage policies remain necessary.

7. Frontend Architecture

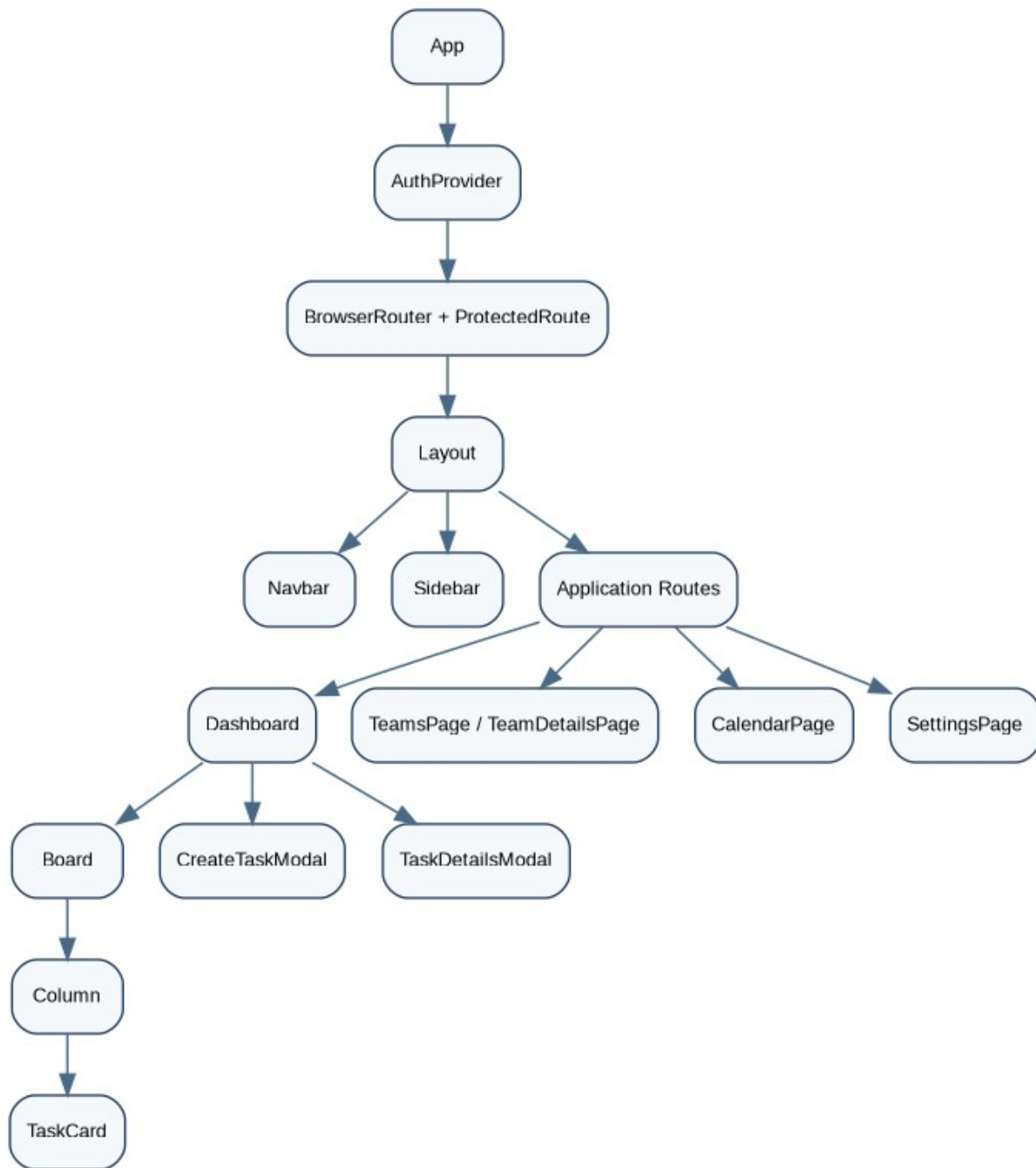


Figure 2. Major React component hierarchy.

The source is organized by feature and responsibility. Authentication components, board components, layout, tasks, teams, shared UI, pages, context, hooks, services, and domain types are separated into dedicated modules. App.tsx acts as the routing composition root and also contains the current home, calendar, and coming-soon page implementations.

7.1 Major Modules

Module	Responsibility
components/auth	Login/sign-up layout and route protection.
components/board	Board, columns, and task cards.
components/tasks	Create and details modals plus task presentation.
components/team	Team pages, modal, types, and database service.
components/layout	Shared sidebar, navbar, and content layout.
context/AuthContext	Session, user, profile, guest state, and avatar operations.
hooks/useTasks	Task state, loading/saving state, and orchestration.
services	Supabase authentication and task data operations.
types	Task and workflow type definitions.
pages	Route-level views for dashboard, status, auth, and settings.

7.2 Component Design Principles

- • Keep route composition separate from reusable domain components.
- • Pass typed callbacks to board and modal components.
- • Centralize backend calls in services rather than duplicating Supabase queries in UI components.
- • Use a shared Layout so navigation, search, and theme controls are consistent.
- • Use modal components to preserve board context while creating or inspecting tasks.

8. Routing and Navigation

Route	Access	Purpose
/login	Public	Email login and guest entry.
/signup	Public	Account registration.
/	Protected	Home summary and navigation entry point.
/tasks	Protected	Full task dashboard.
/tasks/:status	Protected	Status-filtered task view.
/calendar	Protected	Due-date-based calendar.
/teams	Protected	Team list and creation.
/teams/:teamId	Protected	Team details, members, roles, and tasks.
/analytics	Protected	Coming-soon placeholder.
/settings	Protected	Profile and preference settings.
*	Protected fallback	Redirect to home.

BrowserRouter is used with a top-level public authentication branch and a protected wildcard branch. This structure gives the authentication pages a distinct layout while allowing the rest of the application to share Layout, Navbar, and Sidebar.

8.1 Navigation Risks

Client-side route protection is not a security control for backend data. A user can modify client code or issue direct requests; therefore, database policies must independently validate identity, ownership, assignment, and role membership.

9. State Management

GamePlan uses React state and Context rather than a dedicated global state library. AuthContext owns session, user, profile, loading, and authentication errors. useTasks owns the task collection, selected task, create-modal state, loading/saving flags, and task operation handlers. App owns search and theme state.

State	Owner	Persistence
Session and user	AuthContext	Supabase client session persistence.
Profile	AuthContext	profiles table; refreshed after profile changes.
Tasks	useTasks	tasks table; local state mirrors successful writes.
Selected task	useTasks	Ephemeral UI state.
Create modal visibility	useTasks	Ephemeral UI state.
Search query	App	Ephemeral UI state.
Theme	App	localStorage key gameplan-theme.

9.1 Update Pattern

After a successful database mutation, useTasks replaces the corresponding local task or removes it. This avoids a full reload after every change. Loading and saving flags guard duplicate actions and allow disabled controls or progress indicators.

9.2 Scaling Consideration

Context and local state are suitable at the current scale. If the application adds realtime subscriptions, optimistic updates, caching, pagination, and cross-route server state, a query cache such as TanStack Query could reduce custom synchronization logic.

10. Authentication and Profile Management

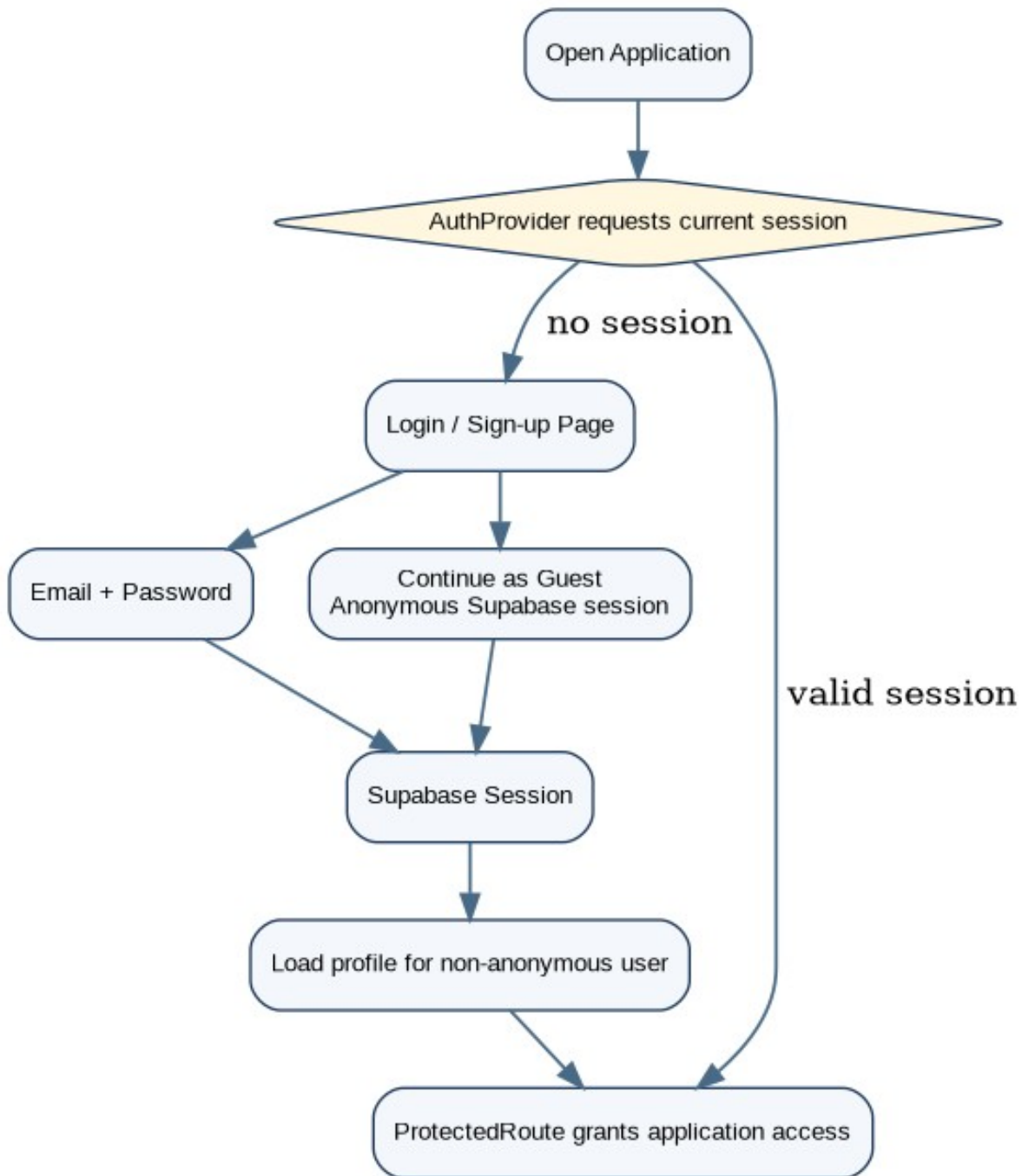


Figure 3. Authentication and session initialization flow.

AuthContext initializes by obtaining the current Supabase session and subscribing to authentication state changes. Permanent users receive a profile loaded from the profiles table. Anonymous users are recognized through the Supabase user is_anonymous flag and intentionally do not load a permanent profile.

10.1 Supported Operations

- • Sign up with full name, email, and password.
- • Sign in with email and password.
- • Sign out.
- • Create an anonymous session.
- • Refresh profile data.
- • Update avatar color.
- • Upload an avatar image to the Avatars bucket.
- • Remove the current avatar object and clear avatar_url.

10.2 Avatar Validation

The context permits image/jpeg, image/png, and image/webp, limits files to 2 MB, chooses a safe extension, and creates a user-scoped storage path. These client checks improve experience but must be paired with Storage policies because client validation can be bypassed.

10.3 Role Model

Application roles are member and supervisor. Unknown or null profile roles map to member in the profile conversion logic. Supervisor status enables review behavior in useTasks. Team memberships also store a role, allowing team-level responsibilities to be represented independently of the global profile role.

11. Task Domain Model and Business Rules

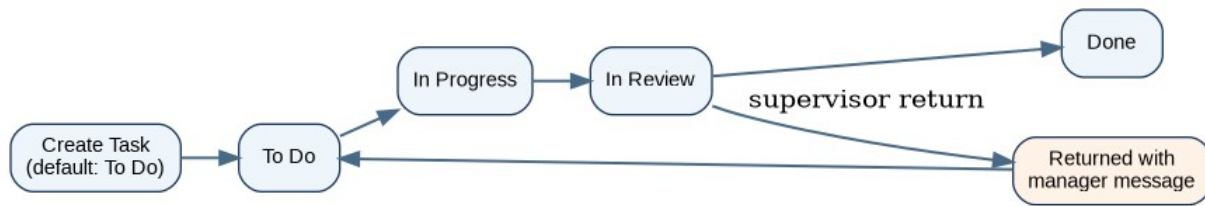


Figure 4. Task lifecycle and supervisor return path.

Field	Meaning
id	Unique task identifier.
title	Required concise description.
description	Long-form task details.
dueDate	Deadline displayed and used for ordering.
isUrgent	Explicit override that sorts ahead of non-urgent tasks.
status	todo, inprogress, inreview, or done.
managerMessage	Feedback when a supervisor returns a task.
userId	Owner/board owner used by authorization logic.
teamId	Optional team association.
assignedTo	Current assignee.
assignedBy	User who assigned the task.
createdBy	Original creator.
createdAt / updatedAt	Audit timestamps.

11.1 Status Mapping

The client uses `inprogress` and `inreview` while the database uses `in_progress` and `in_review`. The task service contains explicit conversion functions in both directions, isolating database naming from UI naming.

11.2 Prioritization

Task retrieval orders rows by `is_urgent` descending, `due_date` ascending with nulls last, and `created_at` ascending. This creates a deterministic queue: urgent tasks first, earlier deadlines next, and older creation time as the final tie-breaker. The UI type also defines deadline colors orange, yellow, and green for relative urgency presentation.

11.3 Review and Return

Moving a task to `In Progress` or `Done` clears a previous manager message. A supervisor may return a task by providing a non-empty message; the service moves it to `To Do` and writes `manager_message`. The return operation also filters by `user_id`, reflecting an owner/supervisor constraint in addition to RLS.

12. Task Data Access and Persistence

`src/services/tasks.ts` is the task data-access layer. It verifies the presence of a Supabase user before each operation and converts between database rows and the client Task model.

Operation	Database behavior
<code>getTasks</code>	Selects all rows permitted by RLS and applies priority ordering.
<code>getTasksByTeamId</code>	Filters by <code>team_id</code> and applies the same ordering.
<code>createTask</code>	Sets owner, assignee default, <code>assigned_by</code> , and <code>created_by</code> to the current user as appropriate.
<code>updateTask</code>	Updates editable fields and returns the saved row.
<code>moveTask</code>	Updates status; clears manager message for selected states.
<code>returnTask</code>	Requires message, moves to todo, and limits by <code>user_id</code> .
<code>deleteTask</code>	Deletes by id and current owner <code>user_id</code> .

12.1 RLS-aware Query Design

Comments in the service deliberately avoid adding `user_id` filters to general read and movement operations because an assigned member may not own the task. The code expects RLS to allow authorized owners and assignees. This is a sound separation only when matching policies are configured in Supabase.

12.2 Consistency

Mutations request the updated row with `select(...).single()`, allowing the UI to use database-generated timestamps and normalized data. Errors are converted into descriptive JavaScript Error messages and propagated to the orchestration layer.

13. Team Collaboration Architecture

Team functionality is implemented through types, service functions, list/detail pages, and a create-team modal. A Team contains metadata and an assembled list of TeamMember records, each joined with a profile representation.

Operation	Behavior
getTeams	Loads teams, memberships, and profiles; assembles nested member lists.
getTeamById	Finds one team from the authorized team set.
getProfilesNotInTeam	Returns profiles not already in a selected team.
createTeam	Creates a team, adds creator as supervisor, adds selected users as members, and rolls back team creation if membership insert fails.
addTeamMember	Adds membership with selected role and added_by.
updateTeamMemberRole	Changes membership role.
removeTeamMember	Removes a user from a team.
deleteTeam	Deletes a team after authorization checks expected from RLS/service constraints.

13.1 Transaction Trade-off

createTeam performs two client-side operations and compensates by deleting the created team if membership insertion fails. This is a practical rollback but not an atomic database transaction. A PostgreSQL function or server-side transaction would provide stronger consistency for production use.

13.2 Role Separation

The team creator is inserted as supervisor. Other selected users default to member. The application supports later role changes, which provides a foundation for differentiated management and contribution permissions.

14. Database Model

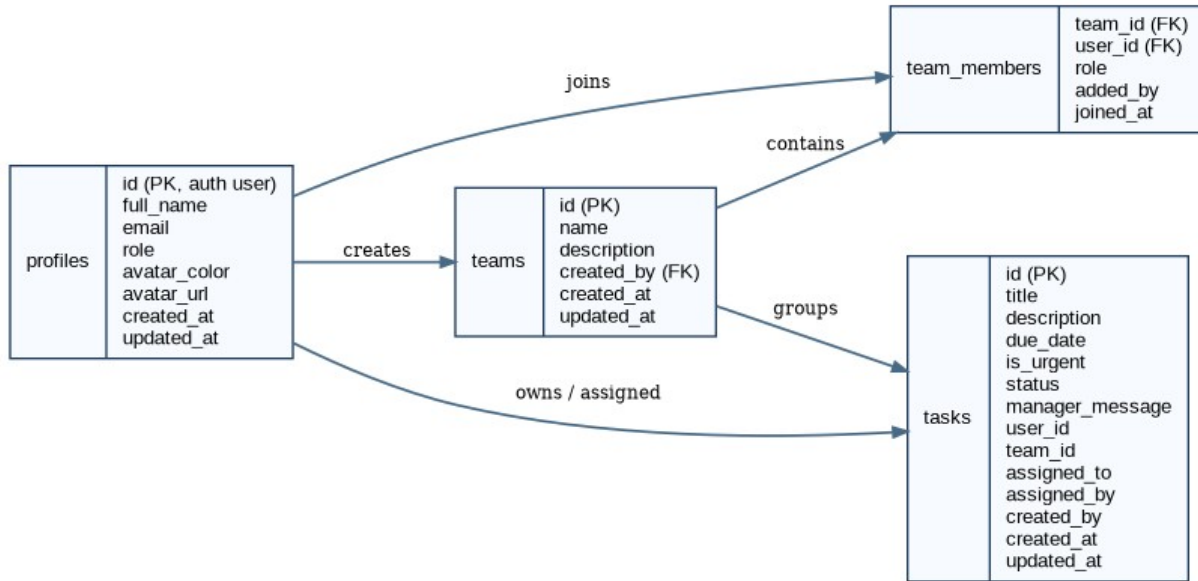


Figure 5. Reconstructed entity relationship model based on source queries.

The repository does not include migration SQL, so the schema below is reconstructed from selected and inserted columns in the TypeScript services. Data types and constraints should be confirmed in the Supabase project before using this as migration documentation.

Table	Observed columns
profiles	id, full_name, email, role, avatar_color, avatar_url, created_at, updated_at
tasks	id, title, description, due_date, is_urgent, status, manager_message, user_id, team_id, assigned_to, assigned_by, created_by, created_at, updated_at
teams	id, name, description, created_by, created_at, updated_at
team_members	team_id, user_id, role, joined_at, added_by (inserted by service)

14.1 Recommended Constraints

- `profiles.id` should reference `auth.users.id`.
- `tasks.status` should be constrained to `todo`, `in_progress`, `in_review`, and `done`.
- `team_members` should have a unique composite key on `team_id` and `user_id`; the service handles PostgreSQL duplicate code 23505.
- Foreign keys should protect team, profile, assignment, and creator relationships.
- Indexes should cover task `team_id`, `assigned_to`, `user_id`, `status`, `due_date`, and urgent ordering.

15. Authorization and Security

Security uses defense in depth: Supabase Authentication establishes identity, ProtectedRoute controls client navigation, role checks control user-interface actions, and RLS is expected to authorize database rows. Only the final layer can protect data from a modified client.

15.1 Authorization Intent

Actor	Expected access
Guest	Anonymous authenticated session; task scope depends on RLS and guest-specific data design.
Member	Read authorized tasks, update permitted assignments, and participate in teams.
Supervisor	Create/assign tasks, review/return work, and manage team membership where authorized.

15.2 Security Gaps to Verify

- No RLS policy SQL is included in the repository.
- No migration history is included, so constraints cannot be independently verified.
- The public profile query in getAvailableProfiles may expose all profiles unless RLS limits rows/columns appropriately.
- Storage bucket policies must limit avatar writes and deletes to the owning user.
- Client role values must never be treated as the sole source of authorization.

15.3 Secret Management

The frontend requires VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY. The anonymous key is intended for public clients but must be combined with RLS. Service-role keys must never be embedded in the client or committed to source control.

16. User Experience and Accessibility

The interface uses a shared navigation layout, task cards, status columns, modals, loading indicators, empty-state behavior, responsive Tailwind utility classes, and light/dark themes. The dashboard includes summary statistics for To Do, urgent, In Progress, and completed work.

16.1 Accessibility Strengths

- Semantic buttons and form controls are used throughout the React source.
- Keyboard shortcuts provide efficient interaction.
- Theme support accommodates user preference and system color scheme.
- Loading and disabled states reduce duplicate submissions.
- Modals support Escape dismissal through a global handler.

16.2 Accessibility Improvements

- Audit focus management when modals open and close.
- Confirm all icon-only controls have accessible names.
- Verify color contrast in both themes.
- Add automated accessibility checks with axe.
- Document keyboard shortcuts inside the settings/help experience.
- Test board navigation without a pointing device.

17. Search, Calendar, Theme, and Keyboard Interaction

17.1 Search

App normalizes the query and filters tasks by title, description, and manager message. Filtering occurs client-side over the loaded task collection and affects dashboard statistics and routed views that receive `filteredTasks`.

17.2 Calendar

`CalendarPage` is implemented inside `App.tsx` and receives filtered tasks. It provides a date-oriented complement to the workflow board. For very large datasets, calendar queries should eventually request only the visible date range rather than loading all tasks.

17.3 Theme

Theme initialization checks `localStorage` first, then the operating-system `prefers-color-scheme` setting. Changes update `localStorage` and the root `colorScheme` property. The D keyboard shortcut toggles the same state.

17.4 Shortcuts

Key	Behavior	Guard
D	Toggle dark/light theme	Ignored while typing.
N	Open create-task modal	Ignored while typing and while tasks are loading/saving.
Escape	Close create modal and selected task	Always active.

18. Error Handling and Reliability

Service methods convert Supabase errors into operation-specific messages. AuthContext exposes `authError`. `useTasks` tracks an error message and protects concurrent task writes with `isSavingTask`. Team services validate user presence and user input before writes.

18.1 Reliability Patterns

- Loading flags prevent premature rendering.
- Saving flags avoid repeated mutations.
- Database writes return canonical saved rows.
- Team creation performs a compensating delete on membership failure.
- Profile upload validates type and size before network operations.
- Authentication state listeners keep UI state synchronized with session changes.

18.2 Recommended Improvements

- Use a shared toast/error boundary for all operation failures.
- Add retry behavior for transient reads.
- Use database transactions for multi-step team creation.
- Introduce structured logging and production error monitoring.
- Define explicit empty/error/loading components for every route.

19. Deployment and Configuration

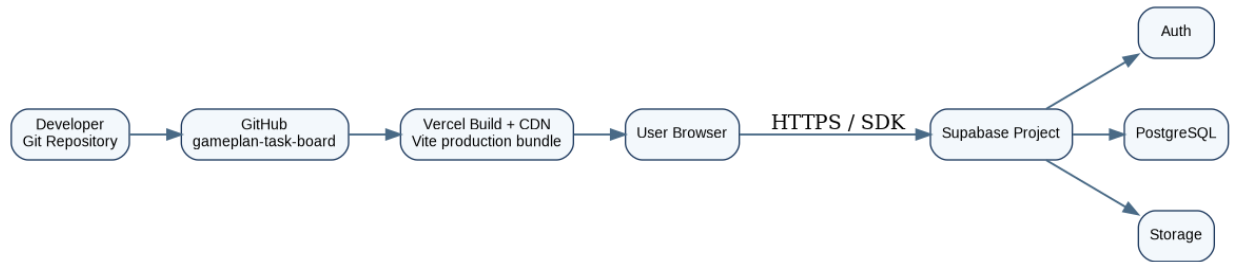


Figure 6. Deployment topology.

The project includes vercel.json and is documented as deployed on Vercel. The frontend is a static Vite application. Browser requests use the Supabase SDK to access the managed backend.

19.1 Local Setup

```

git clone https://github.com/Mananp2207/gameplan-task-board.git
cd gameplan-task-board
npm install
# create .env with the Supabase public client values
npm run dev
  
```

19.2 Environment Variables

Variable	Purpose	Sensitivity
VITE_SUPABASE_URL	Supabase project endpoint.	Public client configuration.
VITE_SUPABASE_ANON_KEY	Supabase anonymous client key.	Public by design; security depends on policies.

19.3 Quality Commands

```

npm run typecheck
npm run lint
npm run build
npm run preview
  
```

The uploaded package does not include node_modules, migrations, or a reproducible test environment. A clean CI pipeline should install dependencies, typecheck, lint, build, and run tests on every pull request.

20. Testing and Quality Strategy

No automated tests are present in the uploaded repository, so this chapter defines the recommended strategy and does not claim completed test coverage.

20.1 Unit Tests

- • Status conversion functions.
- • Task sorting and filtering.
- • Date and deadline color utilities.
- • Profile mapping and anonymous-user detection.
- • Validation helpers and error-message conversion.

20.2 Component Tests

- • ProtectedRoute redirect and loading behavior.
- • CreateTaskModal required fields and default status.
- • TaskDetailsModal actions by role and status.
- • Navbar search and theme controls.
- • Team membership and role controls.

20.3 Integration Tests

- • Email signup creates/loads a profile.
- • Anonymous session can enter the application.
- • Task creation persists and appears in the board.
- • Member moves assigned task through workflow.
- • Supervisor returns a review task with a message.
- • Team creation adds the creator as supervisor.
- • RLS blocks unauthorized reads and writes.

20.4 End-to-End Critical Path

Sign in -> Create team -> Add member -> Create and assign task

-> Member starts task -> Member submits for review

-> Supervisor returns with feedback or completes review -> Task reaches Done

21. Performance and Scalability

The current architecture is appropriate for a portfolio application and small-team workload. The client loads an authorized task collection and filters it locally. Team retrieval uses multiple queries and assembles the relationship graph in memory.

21.1 Current Strengths

- Static frontend delivery through Vercel/CDN.
- Managed PostgreSQL and authentication.
- Indexed relational model can support efficient filtering.
- Mutations update local state instead of reloading the full application.

21.2 Scaling Risks

- Loading all tasks becomes expensive for large accounts.
- Client-side search cannot query unloaded records.
- `getTeams` performs multi-query assembly and may grow with membership volume.
- Realtime updates are not implemented, so concurrent clients may become stale.
- No pagination or virtualized board rendering is present.

21.3 Evolution Path

- Add server-side pagination and date/status filters.
- Use database views or RPC functions for joined team summaries.
- Add targeted indexes based on query plans.
- Introduce Supabase Realtime subscriptions with conflict handling.
- Use a server-state cache and optimistic mutation patterns.
- Move privileged multi-step operations to database functions or edge functions.

22. Design Decisions and Trade-offs

Decision	Benefit	Trade-off
Supabase instead of custom API	Rapid auth/database/storage integration.	Platform coupling and policy configuration become critical.
React Context/local state	Low complexity for current scale.	More manual synchronization as server state grows.
PostgreSQL relational model	Integrity and expressive relationships.	Requires deliberate schema and migrations.
Client-side search	Immediate response and simple implementation.	Limited to loaded tasks.
Compensating rollback for team creation	Simple recovery without server function.	Not fully atomic.
Anonymous guest sessions	Low-friction evaluation.	Requires carefully isolated guest data.
Separate UI and database status names	Friendly client naming.	Requires conversion code and consistency.
Four fixed workflow stages	Clear and teachable process.	Less customizable than configurable boards.

23. Challenges and Lessons Learned

Role-aware workflow

Supporting both Member and Supervisor behavior requires more than hiding buttons. Ownership, assignment, review, return messages, and database policies must agree.

Guest and permanent users

Anonymous sessions improve onboarding but cannot rely on profile data in the same way as permanent accounts. The context explicitly separates these paths.

Database ownership versus assignment

An assignee may update a task they do not own. The service avoids over-filtering queries and delegates final access decisions to RLS.

Multi-step team creation

Creating metadata and memberships across two requests introduces partial-failure risk. The compensating delete demonstrates awareness of consistency, while also revealing the value of transactions.

Keeping UI and persistence synchronized

Returning saved rows from mutations and replacing local objects prevents stale timestamps and duplicated reload logic.

Scope control

Analytics, realtime collaboration, notifications, and full automated testing are valuable, but the source appropriately leaves some as future work rather than hiding incomplete implementation.

24. Known Limitations and Technical Debt

Area	Current limitation	Recommended action
Database reproducibility	No migrations or policy SQL in repository.	Add supabase/migrations and documented seed data.
Testing	No automated test suite.	Add Vitest, React Testing Library, Playwright, and policy tests.
Analytics	Coming-soon route only.	Implement metrics after defining reliable event/data model.
Drag-and-drop	Packages installed but no source integration verified.	Either wire dnd-kit with keyboard support or remove dependencies.
Realtime	No subscriptions.	Add scoped realtime channels and reconcile local state.
Team creation atomicity	Compensating rollback across requests.	Move to a transaction/RPC.
Schema typing	Handwritten database row types.	Generate Supabase database types.
Observability	No production monitoring visible.	Add error tracking and structured logs.
Documentation	Original repository README is Vite template.	Replace with the README delivered in this package.

25. Future Roadmap

Release	Focus	Candidate capabilities
1.1 - Quality Foundation	Reproducibility and confidence	Migrations, RLS policy tests, automated unit/component/E2E tests, CI.
1.2 - Collaboration	Team communication	Comments, activity history, invitations, richer permissions.
1.3 - Awareness	Timely updates	In-app/email notifications and due-date reminders.
1.4 - Realtime	Concurrent teamwork	Supabase Realtime subscriptions and presence.
1.5 - Files and Insights	Supporting materials and reporting	Attachments, analytics dashboard, export.
2.0 - Intelligent Planning	Advanced assistance	AI-assisted suggestions clearly separated from deterministic deadline ordering.

The roadmap deliberately places testing, migrations, and policy verification before headline features. This reduces the risk of scaling an application whose security and data model cannot be reproduced.

26. Developer Guide

26.1 Prerequisites

- • Current Node.js and npm.
- • A Supabase project with Authentication, PostgreSQL tables, Storage bucket, and policies.
- • A local .env file containing public Supabase client configuration.

26.2 Repository Tour

src/

components/auth	authentication UI and route protection
components/board	Board, Column
components/tasks	task cards and modal workflows
components/team	team UI, types, and database service
components/layout	Navbar, Sidebar, Layout
context	AuthContext
hooks	useTasks
pages	route-level pages
services	auth and task data access
types	task domain model
lib	Supabase client initialization

26.3 Adding a Feature

- • Define or update domain types first.
- • Add service operations with explicit error handling.
- • Keep authorization in database policies, not only in UI logic.
- • Add orchestration in a hook/context when state is shared.
- • Build focused components and route composition.
- • Add unit, integration, and policy tests.
- • Update README and this design document.

26.4 Pull Request Checklist

- • Typecheck passes.
- • Lint passes.
- • Production build succeeds.
- • Tests cover new behavior.
- • No secret or service-role key is committed.
- • RLS is tested for allowed and denied actors.
- • Loading, empty, error, light, dark, and responsive states are checked.
- • Documentation is updated.

27. User Guide

27.1 Start a Session

Use the login page to sign in with a permanent account, create a new account, or continue as a guest. Permanent accounts can use stored profile information and avatar features; guest behavior is intentionally restricted by application and database configuration.

27.2 Manage Tasks

- • Open the Tasks dashboard.
- • Select Create New Task or press N.
- • Enter a title, description, due date, and optional urgent flag.
- • For team work, select the team and assignee when available.
- • Open a card to review details and available actions.
- • Move work through To Do, In Progress, In Review, and Done.
- • Supervisors can return review work with a required message.

27.3 Find and Plan Work

- • Use the navbar search to filter title, description, and manager feedback.
- • Use status routes to focus on one workflow stage.
- • Use Calendar to view tasks by due date.
- • Urgent tasks and earlier deadlines are prioritized in retrieval order.

27.4 Preferences

- • Toggle the theme from the navigation or press D.
- • Press Escape to close open task/create experiences.
- • Use Settings for profile and avatar preferences when enabled for the account.

28. Interview Guide

Describe GamePlan in 30 seconds.

GamePlan is a React and TypeScript Kanban task manager backed by Supabase and PostgreSQL. It supports email and anonymous authentication, four-stage task workflows, due-date and urgency ordering, team membership with Member/Supervisor roles, review returns, search, calendar planning, themes, profile customization, and Vercel deployment.

Why Supabase?

It combined authentication, relational persistence, storage, and policy-based authorization without requiring a custom server. That let the project focus on domain workflows while still exposing important backend concepts such as RLS and relational modeling.

Where is the real security boundary?

In Supabase/PostgreSQL policies. ProtectedRoute and role-based UI controls improve experience, but a modified browser can bypass them. RLS and Storage policies must independently validate access.

How are tasks prioritized?

The database query orders urgent tasks first, then due date ascending, then creation time. The urgent flag is an explicit override, while due dates provide automatic ordering.

What is an important trade-off?

Team creation uses two client requests and a compensating delete if membership insertion fails. It is simple, but not atomic. A PostgreSQL function or server transaction would be safer.

What would you improve first?

Add reproducible migrations and RLS policies, automated policy/integration tests, and CI. Those improvements increase confidence before adding realtime, notifications, or analytics.

Does the project have drag-and-drop?

The dependencies are installed, but the uploaded source does not import dnd-kit, so I would not claim it as completed. Task movement is implemented through task actions and service updates.

How would you scale it?

Add server-side pagination/filtering, generated database types, indexed query paths, a server-state cache, database functions for multi-step writes, and scoped realtime subscriptions.

A. Appendix - Repository File Inventory

- • src/App.css
- • src/App.tsx
- • src/assets/hero.png
- • src/assets/react.svg
- • src/assets/vite.svg
- • src/components/auth/AuthLayout.tsx
- • src/components/auth/LoginForm.tsx
- • src/components/auth/ProtectedRoute.tsx
- • src/components/auth/SignUpForm.tsx
- • src/components/board/Board.tsx
- • src/components/board/Column.tsx
- • src/components/dashboard/DashboardStats.tsx
- • src/components/layout/Layout.tsx
- • src/components/layout/Navbar.tsx
- • src/components/layout/Sidebar.tsx
- • src/components/tasks/CreateTaskModal.tsx
- • src/components/tasks/TaskCard.tsx
- • src/components/tasks/TaskDetailsModal.tsx
- • src/components/tasks/task.ts
- • src/components/team/CreateTeamModal.tsx
- • src/components/team/TeamCard.tsx
- • src/components/team/TeamDetailsPage.tsx
- • src/components/team/TeamsPage.tsx
- • src/components/team/team.types.ts
- • src/components/team/team:ID
- • src/components/team/teams.service.ts
- • src/components/ui/Toast.tsx
- • src/context/AuthContext.tsx
- • src/hooks/useTasks.ts
- • src/index.css
- • src/lib/supabase.ts
- • src/main.tsx
- • src/pages/Completed.tsx
- • src/pages/Dashboard.tsx
- • src/pages/InProgress.tsx
- • src/pages/LoginPage.tsx
- • src/pages/Review.tsx
- • src/pages/SettingsPage.tsx
- • src/pages/SignUpPage.tsx
- • src/pages/TaskStatusPage.tsx
- • src/pages/ToDo.tsx
- • src/services/auth.ts

- • src/services/tasks.ts
- • src/types/task.ts

B. Appendix - API and Data Operation Catalog

Module	Function	Purpose
services/auth	signUp, signIn, signInAnonymously, signOut	Authentication operations.
context/AuthContext	refreshProfile, updateAvatarColor, uploadAvatar, removeAvatar	Profile and avatar lifecycle.
services/tasks	getTasks, getTasksByTeamId	Authorized task reads.
services/tasks	createTask, updateTask, moveTask, returnTask, deleteTask	Task mutations.
components/team/teams.service	getTeams, getTeamById, getAvailableProfiles, getProfilesNotInTeam	Team and candidate-member reads.
components/team/teams.service	createTeam, addTeamMember, updateTeamMemberRole, removeTeamMember, deleteTeam	Team mutations.

C. Appendix - Manual Verification Checklist

Authentication

- • ☐ Email sign-up
- • ☐ Email sign-in
- • ☐ Invalid credentials
- • ☐ Anonymous guest
- • ☐ Session restoration
- • ☐ Logout
- • ☐ Protected redirect

Tasks

- • ☐ Create required fields
- • ☐ Urgent ordering
- • ☐ Due-date ordering
- • ☐ Edit
- • ☐ Delete
- • ☐ Each status transition
- • ☐ Supervisor return message
- • ☐ Member authorization denial

Teams

- • ☐ Create team
- • ☐ Creator supervisor membership
- • ☐ Add member
- • ☐ Duplicate member error
- • ☐ Change role
- • ☐ Remove member
- • ☐ Delete team

UX

- • ☐ Search all supported fields
- • ☐ Calendar deadlines
- • ☐ Theme persistence
- • ☐ D/N/Escape shortcuts
- • ☐ Responsive sidebar/navbar
- • ☐ Modal keyboard focus

Security

- • ☐ Unauthorized profile query denied
- • ☐ Unauthorized task read denied
- • ☐ Unauthorized task update/delete denied
- • ☐ Storage upload/delete ownership
- • ☐ Guest isolation

D. Appendix - Suggested RLS Test Matrix

The following is a behavioral test matrix, not policy SQL. It should be converted into database tests after the actual policies are added to source control.

Table / operation	Owner	Assigned member	Unrelated authenticated user	Guest
profiles: select own	Allow	N/A	Deny or restricted public fields	Deny
tasks: select authorized	Allow	Allow when assigned/team-authorized	Deny	Guest-scoped only
tasks: update workflow	Allow	Allow permitted fields/status	Deny	Guest-scoped only
tasks: delete	Allow	Deny unless policy says otherwise	Deny	Guest-scoped only
teams: select	Allow if member	Allow if member	Deny	Deny
team_members: manage	Supervisor	Deny	Deny	Deny
avatars: write/delete	Own path only	Own path only	Deny other paths	Deny

E. Appendix - Glossary

Term	Definition
BaaS	Backend-as-a-Service; managed authentication, database, APIs, and storage.
Kanban	Visual workflow model that organizes work into status columns.
RLS	Row Level Security; PostgreSQL policies evaluated per row and operation.
RBAC	Role-Based Access Control; permissions associated with roles.
Anonymous session	Authenticated Supabase user without a permanent account/profile.
Compensating action	A follow-up operation that attempts to undo a partially completed workflow.
SPA	Single-page application with client-side routing.
CRUD	Create, Read, Update, and Delete operations.
Trust boundary	Point at which data or identity must be independently validated.

F. Appendix - Source-Based Conclusions

The uploaded source supports the conclusion that GamePlan is a meaningful full-stack portfolio project rather than only a visual mock-up. It contains real authentication calls, relational queries, task and team mutations, role-aware review logic, profile/storage operations, protected routing, and deployment configuration.

It also supports an honest discussion of incompleteness: the repository lacks database migrations and automated tests, Analytics is a placeholder, and drag-and-drop is not wired in the source despite installed dependencies. Presenting these limitations clearly makes the project more credible and gives a strong answer to interview questions about technical debt and next steps.

The most valuable next engineering investment is not another visible feature. It is reproducibility and verification: source-controlled schema migrations, RLS and Storage policies, generated database types, automated tests, and continuous integration. Once those foundations are in place, realtime collaboration, notifications, activity history, attachments, and analytics can be added with much lower risk.